Imię	Nazwisko	nr. indeksu
Natan	Jabłoński	155621

Skrypt testujący

In [196…

```
def runTests(testPathPrefix, correctPathPrefix, testExt, correctExt, testImp
    paths = os.listdir(testPathPrefix)[:-1]
    rnaNames = [ name.split(".")[0] for name in paths ]


    test = [ testPathPrefix+name+testExt for name in rnaNames]
    correct = [ correctPathPrefix+name+correctExt for name in rnaNames]


    for testPath, correctPath, rnaName in zip(test, correct, rnaNames):
        if testSpecific is not None and rnaName != testSpecific:
            continue
        testInstance = testImportFun(testPath)
        testCase = processingFun(testInstance)
        correctCase = correctImportFun(correctPath)
        if testCase == correctCase :
            print(f"{rnaName}: correct")
        else:
            print(f"{rnaName}: wrong")
            print("test:")
            print(testCase)
            print("correct:")
            print(correctCase)
```

1. Konwersja dot-bracket do BPSEQ

Wczytywanie plików

In [197…

```
class dotBracket:
    def __init__(self, seq, db):
        self.seq = seq
        self.dbSeq = db

def readDotBracket(path):
    with open(path, "r") as f:
        lines = f.readlines()
    return dotBracket(lines[0][:-1], lines[1][:-1])

def readBPSEQ(path):
    res = []
    with open(path, "r") as f:
        for line in f.readlines():
```

```
            valueList = line.split(" ")
            valueTuple = (int(valueList[0]),
                          valueList[1],
                          int(valueList[2]))
            res.append(valueTuple)
    return res
```

Funkcje do konwersji dbn na bpseq

In [198…

```
import os

def DotBracketToBp(db: dotBracket):
    bp = []

    openingBrackets = ["(", "{", "[", "<", "A", "B"]
    closingBrackets = [")", "}", "]", ">", "a", "b"]
    stacks = [ [] for _ in openingBrackets ]

    for i in range(len(db.dbSeq)):
        if db.dbSeq[i] in openingBrackets:
            stackIdx = openingBrackets.index(db.dbSeq[i])
            stacks[stackIdx].append(i)
        elif db.dbSeq[i] in closingBrackets:
            stackIdx = closingBrackets.index(db.dbSeq[i])
            j = stacks[stackIdx].pop()
            bp.append((j+1, i+1))

    return bp

def DotBracketToBPSEQ(db: dotBracket):
    bp = DotBracketToBp(db)
    bp.extend([ (pair[1], pair[0]) for pair in bp])
    bp = sorted(bp,key=lambda x: x[0])
    if bp == [] :
        res = [ (idx+1, nuc, 0) for idx, nuc in enumerate(db.seq) ]
    else:
        res = []
        bpIdx = 0
        for idx, nuc in enumerate(db.seq):
            currentBpTuple = bp[bpIdx]
            if idx+1 == currentBpTuple[0] :
                res.append((currentBpTuple[0], nuc, currentBpTuple[1]))
                if bpIdx != len(bp)-1:
                    bpIdx += 1
            else :
                res.append((idx+1,nuc,0))
    return res
```

Test poprawności

In [200…

```
runTests(testPathPrefix = "dot-brackets/",
         correctPathPrefix="dot-brackets-correct/correct/3.0/",
```

```
         testExt=".dbn",
         correctExt=".bpseq",
         testImportFun=readDotBracket,
         correctImportFun=readBPSEQ,
         processingFun=DotBracketToBPSEQ)
```

```
6ZM6-AA: correct
5SWD-B: correct
6ERI-BA: correct
3Q1Q-B: correct
6UES-A: correct
6V3A-sN1: correct
5TPY-A: correct
3IVN-A: correct
4FRN-B: correct
5J7L-AA: correct
6TPQ-U: correct
6N5P-A: correct
5O60-A: correct
3Q3Z-V: correct
7K16-P: correct
6AGB-A: correct
6NEQ-A: correct
3IGI-A: correct
6GAZ-AA: correct
1ET4-A: correct
6HA1-a: correct
```

Znajdowanie motywu spinki do włosów

Odczytywanie wyników

In [201…

```
def readCorrectHairpin(path):
    res = []
    with open(path,"r") as f:
        for line in f.readlines():
            values = line[:-1].split("-")
            res.append((int(values[0]), values[1], int(values[2])))
    return res


testCorrect = readCorrectHairpin("dot-brackets-correct/correct/3.5/7K16-P.ha
print(testCorrect)
```

```
[(13, 'CGAAAG', 18)]
```

Szukanie motywu spinki do włosów

In [202…

```
import numpy as np
test = readBPSEQ("dot-brackets-correct/correct/3.0/7K16-P.bpseq")

def findHairpins(bpseq):
```

```
    hairpins = []
    startValues = np.array([ values[0] for values in bpseq])
    nucValues = np.array([ values[1] for values in bpseq])
    bondValues = np.array([ values[2] for values in bpseq])
    for values in bpseq:
        idx = values[0]
        nuc = values[1]
        bond = values[2]
        if bond > idx + 1 and (bondValues[idx:bond-1] == 0).all():
            hairpins.append((idx, "".join(nucValues[idx-1:bond]),bond))
    return hairpins

print(findHairpins(test))
print(testCorrect)
print(f"initial test passed: {findHairpins(test) == testCorrect}")
```

```
[(13, 'CGAAAG', 18)]
[(13, 'CGAAAG', 18)]
initial test passed: True
```

Test poprawności

In [203…

```
runTests(testPathPrefix="dot-brackets-correct/correct/3.0/",
        correctPathPrefix="dot-brackets-correct/correct/3.5/",
        testExt=".bpseq",
        correctExt=".hairpins",
        testImportFun=readBPSEQ,
        correctImportFun=readCorrectHairpin,
        processingFun=findHairpins)
```

```
7K16-P: correct
6UES-A: correct
6NEQ-A: correct
3Q1Q-B: correct
6TPQ-U: correct
3Q3Z-V: correct
1ET4-A: correct
6ERI-BA: correct
5O60-A: correct
3IVN-A: correct
6N5P-A: correct
6HA1-a: correct
6V3A-sN1: correct
6AGB-A: correct
6ZM6-AA: correct
5J7L-AA: correct
5SWD-B: correct
4FRN-B: correct
5TPY-A: correct
6GAZ-AA: correct
3IGI-A: correct
```

Szukanie motywu stem

odczytywanie wyników

In [204…

```
def readCorrectStem(path):
    res = []
    with open(path,"r") as f:
        for line in f.readlines():
            values = line[:-1].split(" ")
            seq1 = values[0].split("-")
            seq2 = values[1].split("-")
            res.append(((int(seq1[0]), seq1[1], int(seq1[2])),
                        (int(seq2[0]), seq2[1], int(seq2[2]))))
    return res


testCorrect = readCorrectStem("dot-brackets-correct/correct/4.0/1ET4-A.stems
for stem in testCorrect:
    print(stem)
```

```
((7, 'GGU', 9), (22, 'CCA', 20))
((11, 'C', 11), (30, 'G', 30))
((12, 'GC', 13), (33, 'CG', 32))
((18, 'C', 18), (28, 'G', 28))
((19, 'C', 19), (26, 'G', 26))
```

Szukanie motywu stemu

In [205…

```
def findSingleStem(start, bpseq):
    startingValues = bpseq[start]
    startIdx = startingValues[0]
    startNuc = startingValues[1]
    startBond = startingValues[2]


    # if startIdx == startBond-1
    #     return ((startIdx, startNuc, startIdx), (startBond, bpseq[startBon

    startValues = np.array([ values[0] for values in bpseq])
    nucValues = np.array([ values[1] for values in bpseq])
    bondValues = np.array([ values[2] for values in bpseq])

    shift = 0
    for iterNum, i in enumerate(range(startBond-2, startIdx-1,-1)):
        values = bpseq[i]
        idx = values[0]
        nuc = values[1]
        bond = values[2]

        if bond == 0 or bond < startIdx:
            break
```

```
        elif bond == startIdx + iterNum + 1:
            shift += 1
    return ((startIdx, "".join(nucValues[start:start+shift+1]),startIdx+shif
            (startBond, "".join(nucValues[startBond-1-shift:startBond][::-1]

def findStems(bpseq):
    activeShift = 0
    stems = []
    for start, values in enumerate(bpseq):
        if activeShift != 0:
            activeShift -= 1
            continue
        idx = values[0]
        nuc = values[1]
        bond = values[2]
        if bond > idx :
            stem = findSingleStem(start, bpseq)
            activeShift = stem[0][2] - stem[0][0]
            stems.append(stem)
    return stems
```

Test poprawności

In [206…

```
runTests(testPathPrefix="dot-brackets-correct/correct/3.0/",
         correctPathPrefix="dot-brackets-correct/correct/4.0/",
         testExt=".bpseq",
         correctExt=".stems",
         testImportFun=readBPSEQ,
         correctImportFun=readCorrectStem,
         processingFun=findStems,
         testSpecific=None)
```

```
7K16-P: correct
6UES-A: correct
6NEQ-A: correct
3Q1Q-B: correct
6TPQ-U: correct
3Q3Z-V: correct
1ET4-A: correct
6ERI-BA: correct
5O60-A: correct
3IVN-A: correct
6N5P-A: correct
6HA1-a: correct
6V3A-sN1: correct
6AGB-A: correct
6ZM6-AA: correct
5J7L-AA: correct
5SWD-B: correct
4FRN-B: correct
5TPY-A: correct
6GAZ-AA: correct
3IGI-A: correct
```

Szukanie motywu single strand

Odczytywanie wyników

In [207…

```
def readCorrectSingle(path):
    res = []
    with open(path,"r") as f:
        for line in f.readlines():
            values = line[:-1].split("-")
            res.append((int(values[0]), values[1], int(values[2])))
    return res


# testCorrect = readCorrectSingle("dot-brackets-correct/correct/4.5/1ET4-A.s
testCorrect = readCorrectSingle("dot-brackets-correct/correct/4.5/7K16-P.str
print(testCorrect)
```

```
[(1, 'GG', 2), (2, 'GCAAG', 6), (27, 'GCUUG', 31), (33, 'GAACC', 37), (42,
'CUC', 44), (45, 'CCCAC', 49), (49, 'CUC', 51)]
```

Funckja szukająca motywu single strand

In [208…

```
def findSingleStrand(bpseq):
    startValues = np.array([ values[0] for values in bpseq])
    nucValues = np.array([ values[1] for values in bpseq])
    bondValues = np.array([ values[2] for values in bpseq])

    singleStrands = []

    startIdx = None
```

```
    for values in bpseq:
        idx = values[0]
        nuc = values[1]
        bond = values[2]
        if idx == len(bpseq) and startIdx != -1:
            singleStrands.append((startIdx, "".join(nucValues[startIdx-1:idx
            continue
        if bond == 0:
            continue
        if bond !=0 and startIdx is None and idx != 1:
            singleStrands.append((1, "".join(nucValues[0:idx]), idx))
            startIdx = idx if idx != len(bpseq) and bpseq[idx][2] == 0 else
            continue
        if bond != 0 and (startIdx == -1 or idx == 1) and (idx != len(bpseq)
            startIdx = idx
            continue
        if bond == startIdx:
            startIdx = idx if (idx != len(bpseq) and bpseq[idx][2] == 0) els
            continue
        if bond != startIdx and startIdx != -1 and startIdx is not None:
            singleStrands.append((startIdx, "".join(nucValues[startIdx-1:idx
            startIdx = idx if idx != len(bpseq) and bpseq[idx][2] == 0 else
            continue


    return singleStrands
```

Test poprawności

In [209…

```
runTests(testPathPrefix="dot-brackets-correct/correct/3.0/",
         correctPathPrefix="dot-brackets-correct/correct/4.5/",
         testExt=".bpseq",
         correctExt=".strands",
         testImportFun=readBPSEQ,
         correctImportFun=readCorrectSingle,
         processingFun=findSingleStrand,
         testSpecific=None)
```

```
7K16-P: wrong
test:
[(1, 'GG', 2), (2, 'GCAAG', 6), (27, 'GCUUG', 31), (33, 'GAACC', 37), (42,
'CUC', 44), (45, 'CCCAC', 49)]
correct:
[(1, 'GG', 2), (2, 'GCAAG', 6), (27, 'GCUUG', 31), (33, 'GAACC', 37), (42,
'CUC', 44), (45, 'CCCAC', 49), (49, 'CUC', 51)]
6UES-A: wrong
test:
[(1, 'GG', 2), (4, 'CAUGAG', 9), (9, 'GUG', 11), (16, 'CGUCAA', 21), (22, 'G
CC', 24), (30, 'UUG', 32), (42, 'AAC', 44), (49, 'CAA', 51), (57, 'GUG', 5
9), (71, 'GUGAUG', 76), (79, 'CAGG', 82), (85, 'GAG', 87), (103, 'CGC', 10
5), (108, 'CAAGCG', 113), (116, 'GGU', 118), (118, 'UC', 119)]
correct:
[(1, 'GGUC', 4), (4, 'CAUGAG', 9), (9, 'GUG', 11), (16, 'CGUCAA', 21), (22,
'GCC', 24), (30, 'UUG', 32), (42, 'AAC', 44), (49, 'CAA', 51), (57, 'GUG', 5
9), (71, 'GUGAUG', 76), (79, 'CAGG', 82), (85, 'GAG', 87), (103, 'CGC', 10
5), (108, 'CAAGCG', 113), (116, 'GGU', 118), (118, 'UC', 119)]
6NEQ-A: correct
3Q1Q-B: wrong
test:
[(1, 'GG', 2), (10, 'CAGG', 13), (18, 'CGC', 20), (38, 'CUU', 40), (44, 'GA
G', 46), (46, 'GGAAA', 50), (51, 'GUC', 53), (66, 'CGG', 68), (103, 'GACAG',
107), (111, 'CAUAGAGAAGAAGAC', 125), (148, 'CAAG', 151), (152, 'GGUGGAACG',
160), (179, 'AGC', 181), (198, 'GGCU', 201), (201, 'UUG', 203), (205, 'CAA
C', 208), (220, 'GCAAG', 224), (227, 'CAAGCAG', 233), (237, 'GUUG', 240), (2
42, 'GUCG', 245), (249, 'CCC', 251), (253, 'UAUUC', 257), (261, 'CGGGUUG', 2
67), (270, 'CGCUUGAG', 277), (296, 'CCCAGAUUGAUG', 307), (312, 'CCCACGAC', 3
19), (319, 'CAGAAU', 324), (330, 'UUAUG', 334), (343, 'CCGUG', 347)]
correct:
[(1, 'GGAGAGGAGC', 10), (10, 'CAGG', 13), (18, 'CGC', 20), (38, 'CUU', 40),
(44, 'GAG', 46), (46, 'GGAAA', 50), (51, 'GUC', 53), (66, 'CGG', 68), (103,
'GACAG', 107), (111, 'CAUAGAGAAGAAGAC', 125), (148, 'CAAG', 151), (152, 'GGU
GGAACG', 160), (179, 'AGC', 181), (198, 'GGCU', 201), (201, 'UUG', 203), (20
5, 'CAAC', 208), (220, 'GCAAG', 224), (227, 'CAAGCAG', 233), (237, 'GUUG', 2
40), (242, 'GUCG', 245), (249, 'CCC', 251), (253, 'UAUUC', 257), (261, 'CGGG
UUG', 267), (270, 'CGCUUGAG', 277), (296, 'CCCAGAUUGAUG', 307), (312, 'CCCAC
GAC', 319), (319, 'CAGAAU', 324), (330, 'UUAUG', 334), (343, 'CCGUG', 347)]
6TPQ-U: wrong
test:
[(1, 'UU', 2), (6, 'UUAGAAAG', 13), (22, 'GUGGAUG', 28), (30, 'CUUG', 33),
(45, 'CGAUGAAG', 52), (57, 'GGACGAA', 63), (68, 'GAUAU', 72), (72, 'UGC', 7
4), (79, 'GGGAG', 83), (87, 'UAAGCAA', 93), (95, 'CUUUGAUC', 102), (107, 'GA
UUU', 111), (113, 'CGAAUG', 118), (147, 'UAU', 149), (173, 'GAG', 175), (17
6, 'AAG', 178), (179, 'GCAGAC', 184), (189, 'GGAACUG', 195), (195, 'GAAAC',
199), (199, 'CAUC', 202), (202, 'CUAAGUAC', 209), (213, 'GAG', 215), (215,
'GGAAGAGAAAG', 225), (232, 'CGAUU', 236), (239, 'CUGAGUAG', 246), (247, 'CG
G', 249), (249, 'GCGAG', 253), (254, 'CGAAACG', 260), (263, 'AUCAG', 267),
(270, 'CAAAC', 274), (293, 'GGGUUG', 298), (298, 'GUA', 300), (308, 'CUGUAC
G', 314), (317, 'GUUACAAAGG', 326), (334, 'UAG', 336), (338, 'UGAAG', 342),
(342, 'GAG', 344), (365, 'GGA', 367), (369, 'GUAACAG', 375), (379, 'UGUAG',
383), (385, 'CAAAA', 389), (397, 'CUC', 399), (402, 'CUG', 404), (408, 'GGA
U', 411), (415, 'GAGUAC', 420), (427, 'ACAC', 430), (430, 'CGUGAAAU', 437),
(444, 'GGAAUC', 449), (449, 'CCG', 451), (456, 'GAC', 458), (466, 'CAAG', 46
9), (471, 'CUAAAUAC', 478), (479, 'UCC', 481), (487, 'GAC', 489), (490, 'CGA
UAGUGAACCAGUAC', 506), (513, 'GGAAAG', 518), (519, 'GUGAAAAG', 526), (526,
```

```
'GCAC', 529), (540, 'GAGUGAAAGAGAUC', 553), (554, 'CUGAAAC', 560), (569, 'UA
CAAGU', 575), (575, 'UAG', 577), (580, 'AGAG', 583), (586, 'CGUU', 589), (58
9, 'UAACG', 593), (596, 'UGAU', 599), (602, 'CGU', 604), (604, 'UGC', 606),
(606, 'CCU', 608), (617, 'AUG', 619), (619, 'GAA', 621), (626, 'CGAG', 629),
(629, 'GUU', 631), (637, 'CCC', 639), (643, 'CAAG', 646), (648, 'UUAAG', 65
2), (662, 'CGGAG', 666), (668, 'CGCAG', 672), (679, 'CGAG', 682), (694, 'CGC
AUGAG', 701), (705, 'GUG', 707), (713, 'AGAC', 716), (717, 'CCG', 719), (71
9, 'GAAAC', 723), (728, 'UGAUCUA', 734), (743, 'CAG', 745), (748, 'UGAAG', 7
52), (769, 'UGGAG', 773), (777, 'CGAAC', 781), (783, 'CAC', 785), (789, 'CGU
UGAAAA', 797), (805, 'GAUG', 808), (808, 'GAG', 810), (818, 'UAG', 820), (82
0, 'GGG', 822), (832, 'CAAUC', 836), (836, 'CGAA', 839), (844, 'GAGAUAGC', 8
51), (851, 'CUG', 853), (855, 'UUC', 857), (862, 'CGAAAU', 867), (889, 'GUAA
G', 893), (901, 'GAGGUAG', 907), (909, 'GCACU', 913), (920, 'ACUAGGGG', 92
7), (939, 'UUACCGAAU', 947), (954, 'AAAC', 957), (958, 'UCC', 960), (960, 'C
GAAUGC', 966), (969, 'AUG', 971), (975, 'UAUC', 978), (984, 'GAG', 986), (98
7, 'UCAG', 990), (996, 'GAG', 998), (1007, 'UCC', 1009), (1015, 'CGAAAG', 10
20), (1031, 'CAGAC', 1035), (1038, 'CCA', 1040), (1042, 'CUAAG', 1046), (105
1, 'CAAAG', 1055), (1055, 'GUA', 1057), (1062, 'UUAAGUG', 1068), (1068, 'GGA
AAAG', 1074), (1075, 'GAUG', 1078), (1087, 'CUUAGAC', 1093), (1093, 'CAA', 1
095), (1099, 'GGA', 1101), (1102, 'UGUUG', 1106), (1120, 'CACCA', 1124), (11
25, 'UUUAAAG', 1131), (1131, 'GAGUG', 1135), (1144, 'CUC', 1146), (1146, 'CA
C', 1148), (1152, 'UCGAG', 1156), (1165, 'CGC', 1167), (1168, 'CGAAAAUGUACC
G', 1180), (1182, 'GGC', 1184), (1184, 'CUAAA', 1188), (1197, 'CGAAG', 120
1), (1224, 'UGGUAG', 1229), (1241, 'AAGGG', 1245), (1246, 'CUGUGAAG', 1253),
(1257, 'GAC', 1259), (1272, 'UGGAG', 1276), (1277, 'CGCUU', 1281), (1285, 'A
GU', 1287), (1288, 'GAGAAUG', 1294), (1300, 'AUGAGUAG', 1307), (1308, 'CGAAA
G', 1313), (1335, 'CGAAUG', 1340), (1342, 'CUAAG', 1346), (1347, 'GUUU', 135
0), (1356, 'GGAAG', 1360), (1368, 'CGC', 1370), (1375, 'GGUUAG', 1380), (138
5, 'GAC', 1387), (1388, 'CUAAG', 1392), (1394, 'CGAGG', 1398), (1407, 'CGUA
G', 1411), (1413, 'CGAUG', 1417), (1418, 'GACAA', 1422), (1446, 'UCC', 144
8), (1452, 'CCAUUUGAG', 1460), (1461, 'CAAUG', 1465), (1468, 'GGGACG', 147
3), (1476, 'GGAG', 1479), (1481, 'AUAG', 1484), (1486, 'GUA', 1488), (1488,
'AAG', 1490), (1492, 'GCGGUAUUG', 1500), (1500, 'GGAUAUCC', 1507), (1509, 'C
GU', 1511), (1512, 'CCAAGC', 1517), (1517, 'CAGUUA', 1522), (1523, 'GGC', 15
25), (1525, 'CUGG', 1528), (1533, 'UAG', 1535), (1548, 'CCC', 1550), (1550,
'CAUAAG', 1555), (1555, 'GGC', 1557), (1558, 'UGAGCUG', 1564), (1564, 'GUGAU
G', 1569), (1571, 'CGAG', 1574), (1575, 'CGAA', 1578), (1585, 'UAG', 1587),
(1588, 'CGAAG', 1592), (1593, 'UUC', 1595), (1597, 'UGAUU', 1601), (1603, 'C
ACAC', 1607), (1614, 'GAAAAGC', 1620), (1623, 'CUAG', 1626), (1627, 'CGAGG',
1631), (1644, 'GUA', 1646), (1648, 'CGCAAAC', 1654), (1663, 'GUAG', 1666),
(1668, 'CGA', 1670), (1682, 'UAAG', 1685), (1688, 'GAUC', 1691), (1694, 'GAG
AAC', 1699), (1706, 'UAA', 1708), (1709, 'GGAA', 1712), (1712, 'ACUCGG', 171
7), (1717, 'GCAAAAUGA', 1725), (1729, 'CGUAAC', 1734), (1734, 'CUU', 1736),
(1736, 'UCGG', 1739), (1739, 'GGAGAAG', 1745), (1754, 'UGUUAG', 1759), (177
0, 'CGAGA', 1774), (1783, 'GUGAAUAGG', 1791), (1799, 'GAC', 1801), (1801, 'C
UG', 1803), (1815, 'CACAG', 1819), (1825, 'UGCGAAG', 1831), (1840, 'CGAAGUAU
A', 1848), (1854, 'UGA', 1856), (1860, 'CUG', 1862), (1864, 'CCGG', 1867),
(1873, 'GAAG', 1876), (1878, 'UUAAG', 1882), (1883, 'AGGAG', 1887), (1891,
'UUAG', 1894), (1901, 'CGAAG', 1905), (1909, 'CGAAU', 1913), (1914, 'UGAAG',
1918), (1925, 'UAAA', 1928), (1951, 'CCUAAGGUAG', 1960), (1961, 'CGA', 196
3), (1970, 'UUG', 1972), (1988, 'CCCG', 1991), (1991, 'GCAC', 1994), (1994,
'CGAAAG', 1999), (2002, 'GCAAC', 2006), (2007, 'GAU', 2009), (2015, 'CAC', 2
017), (2017, 'CUGU', 2020), (2022, 'UCA', 2024), (2031, 'GAC', 2033), (2038,
'UGAAAUU', 2044), (2045, 'AUA', 2047), (2047, 'AGUA', 2050), (2054, 'GUGAAG
A', 2060), (2061, 'UGC', 2063), (2067, 'UUAC', 2070), (2077, 'CAG', 2079),
(2081, 'ACG', 2083), (2084, 'GAAAGACC', 2091), (2094, 'GUG', 2096), (2101,
```

```
'UUUA', 2104), (2108, 'CAG', 2110), (2118, 'UUG', 2120), (2122, 'AUG', 212
4), (2125, 'UUGGUA', 2130), (2134, 'CUUG', 2137), (2137, 'GUACAGG', 2143),
(2143, 'GAUAG', 2147), (2148, 'GUA', 2150), (2150, 'AGGAG', 2154), (2154, 'G
CC', 2156), (2156, 'CUUGGAAACCGGAGC', 2170), (2175, 'GCUUUGGUGGAGG', 2187),
(2188, 'CAUC', 2191), (2191, 'CGGUGGGAUACU', 2202), (2202, 'UAC', 2204), (22
06, 'CUG', 2208), (2208, 'GGC', 2210), (2213, 'UAUUGA', 2218), (2219, 'CCU',
2221), (2223, 'CUAACC', 2228), (2229, 'CGC', 2231), (2249, 'GAGACAG', 2255),
(2263, 'UGGG', 2266), (2270, 'UUUG', 2273), (2275, 'CUGGGGCG', 2282), (2291,
'CUAAAAGGUAAC', 2302), (2312, 'CAAAG', 2316), (2322, 'CUC', 2324), (2324, 'C
AG', 2326), (2344, 'CGCAG', 2348), (2348, 'GAG', 2350), (2353, 'UAA', 2355),
(2359, 'CACAAG', 2364), (2370, 'UUGAC', 2374), (2377, 'CGAG', 2380), (2382,
'CCUAC', 2386), (2386, 'CAAG', 2389), (2391, 'CGAG', 2394), (2397, 'GGG', 23
99), (2408, 'CGG', 2410), (2414, 'UAG', 2416), (2416, 'GUGAUCC', 2422), (242
8, 'UUC', 2430), (2431, 'CGCA', 2434), (2437, 'GAAG', 2440), (2448, 'GCUCAA
C', 2454), (2455, 'GGAUAAAA', 2462), (2465, 'UACC', 2468), (2473, 'GGAUAAC
A', 2480), (2484, 'UUA', 2486), (2486, 'AUC', 2488), (2493, 'CCAAG', 2497),
(2507, 'CGACG', 2511), (2516, 'GGUUU', 2520), (2523, 'CAC', 2525), (2525, 'C
CU', 2527), (2527, 'UCGAUGUC', 2534), (2537, 'CUC', 2539), (2544, 'CAUC', 25
47), (2567, 'CAAG', 2570), (2572, 'GUU', 2574), (2588, 'AUUAAAG', 2594), (25
98, 'UACG', 2601), (2601, 'GCGAG', 2605), (2607, 'UGG', 2609), (2609, 'GGUUC
AG', 2615), (2628, 'CAG', 2630), (2633, 'CGGU', 2636), (2636, 'UCC', 2638),
(2639, 'CUAU', 2642), (2653, 'CGCAG', 2657), (2663, 'UUGAG', 2667), (2670,
'GAGC', 2673), (2679, 'UUAGUAC', 2685), (2690, 'GGACCG', 2695), (2701, 'GA
C', 2703), (2707, 'CCGCU', 2711), (2714, 'UGUAC', 2718), (2728, 'UGC', 273
0), (2730, 'CCAA', 2733), (2738, 'AUCG', 2741), (2745, 'GGUAG', 2749), (275
2, 'AUG', 2754), (2758, 'GGACG', 2762), (2764, 'GAUAAG', 2769), (2772, 'CUGA
AAG', 2778), (2778, 'GCAUCUAAG', 2786), (2789, 'UGAAGC', 2794), (2796, 'CC
C', 2798), (2801, 'CAAG', 2804), (2804, 'GAUGAG', 2809), (2816, 'CAU', 281
8), (2829, 'AGUAAG', 2834), (2854, 'GUUGAU', 2859), (2866, 'GAG', 2868), (28
70, 'UGGAAG', 2875), (2888, 'UGGAG', 2892), (2894, 'UGAC', 2897), (2901, 'UA
C', 2903), (2905, 'AAUCG', 2909), (2912, 'CGAGG', 2916), (2922, 'ACC', 292
4)]
correct:
[(1, 'UUAAGU', 6), (6, 'UUAGAAAG', 13), (22, 'GUGGAUG', 28), (30, 'CUUG', 3
3), (45, 'CGAUGAAG', 52), (57, 'GGACGAA', 63), (68, 'GAUAU', 72), (72, 'UG
C', 74), (79, 'GGGAG', 83), (87, 'UAAGCAA', 93), (95, 'CUUUGAUC', 102), (10
7, 'GAUUU', 111), (113, 'CGAAUG', 118), (147, 'UAU', 149), (173, 'GAG', 17
5), (176, 'AAG', 178), (179, 'GCAGAC', 184), (189, 'GGAACUG', 195), (195, 'G
AAAC', 199), (199, 'CAUC', 202), (202, 'CUAAGUAC', 209), (213, 'GAG', 215),
(215, 'GGAAGAGAAAG', 225), (232, 'CGAUU', 236), (239, 'CUGAGUAG', 246), (24
7, 'CGG', 249), (249, 'GCGAG', 253), (254, 'CGAAACG', 260), (263, 'AUCAG', 2
67), (270, 'CAAAC', 274), (293, 'GGGUUG', 298), (298, 'GUA', 300), (308, 'CU
GUACG', 314), (317, 'GUUACAAAGG', 326), (334, 'UAG', 336), (338, 'UGAAG', 34
2), (342, 'GAG', 344), (365, 'GGA', 367), (369, 'GUAACAG', 375), (379, 'UGUA
G', 383), (385, 'CAAAA', 389), (397, 'CUC', 399), (402, 'CUG', 404), (408,
'GGAU', 411), (415, 'GAGUAC', 420), (427, 'ACAC', 430), (430, 'CGUGAAAU', 43
7), (444, 'GGAAUC', 449), (449, 'CCG', 451), (456, 'GAC', 458), (466, 'CAA
G', 469), (471, 'CUAAAUAC', 478), (479, 'UCC', 481), (487, 'GAC', 489), (49
0, 'CGAUAGUGAACCAGUAC', 506), (513, 'GGAAAG', 518), (519, 'GUGAAAAG', 526),
(526, 'GCAC', 529), (540, 'GAGUGAAAGAGAUC', 553), (554, 'CUGAAAC', 560), (56
9, 'UACAAGU', 575), (575, 'UAG', 577), (580, 'AGAG', 583), (586, 'CGUU', 58
9), (589, 'UAACG', 593), (596, 'UGAU', 599), (602, 'CGU', 604), (604, 'UGC',
606), (606, 'CCU', 608), (617, 'AUG', 619), (619, 'GAA', 621), (626, 'CGAG',
629), (629, 'GUU', 631), (637, 'CCC', 639), (643, 'CAAG', 646), (648, 'UUAA
G', 652), (662, 'CGGAG', 666), (668, 'CGCAG', 672), (679, 'CGAG', 682), (69
4, 'CGCAUGAG', 701), (705, 'GUG', 707), (713, 'AGAC', 716), (717, 'CCG', 71
```

```
9), (719, 'GAAAC', 723), (728, 'UGAUCUA', 734), (743, 'CAG', 745), (748, 'UG
AAG', 752), (769, 'UGGAG', 773), (777, 'CGAAC', 781), (783, 'CAC', 785), (78
9, 'CGUUGAAAA', 797), (805, 'GAUG', 808), (808, 'GAG', 810), (818, 'UAG', 82
0), (820, 'GGG', 822), (832, 'CAAUC', 836), (836, 'CGAA', 839), (844, 'GAGAU
AGC', 851), (851, 'CUG', 853), (855, 'UUC', 857), (862, 'CGAAAU', 867), (88
9, 'GUAAG', 893), (901, 'GAGGUAG', 907), (909, 'GCACU', 913), (920, 'ACUAGGG
G', 927), (939, 'UUACCGAAU', 947), (954, 'AAAC', 957), (958, 'UCC', 960), (9
60, 'CGAAUGC', 966), (969, 'AUG', 971), (975, 'UAUC', 978), (984, 'GAG', 98
6), (987, 'UCAG', 990), (996, 'GAG', 998), (1007, 'UCC', 1009), (1015, 'CGAA
AG', 1020), (1031, 'CAGAC', 1035), (1038, 'CCA', 1040), (1042, 'CUAAG', 104
6), (1051, 'CAAAG', 1055), (1055, 'GUA', 1057), (1062, 'UUAAGUG', 1068), (10
68, 'GGAAAAG', 1074), (1075, 'GAUG', 1078), (1087, 'CUUAGAC', 1093), (1093,
'CAA', 1095), (1099, 'GGA', 1101), (1102, 'UGUUG', 1106), (1120, 'CACCA', 11
24), (1125, 'UUUAAAG', 1131), (1131, 'GAGUG', 1135), (1144, 'CUC', 1146), (1
146, 'CAC', 1148), (1152, 'UCGAG', 1156), (1165, 'CGC', 1167), (1168, 'CGAAA
AUGUACCG', 1180), (1182, 'GGC', 1184), (1184, 'CUAAA', 1188), (1197, 'CGAA
G', 1201), (1224, 'UGGUAG', 1229), (1241, 'AAGGG', 1245), (1246, 'CUGUGAAG',
1253), (1257, 'GAC', 1259), (1272, 'UGGAG', 1276), (1277, 'CGCUU', 1281), (1
285, 'AGU', 1287), (1288, 'GAGAAUG', 1294), (1300, 'AUGAGUAG', 1307), (1308,
'CGAAAG', 1313), (1335, 'CGAAUG', 1340), (1342, 'CUAAG', 1346), (1347, 'GUU
U', 1350), (1356, 'GGAAG', 1360), (1368, 'CGC', 1370), (1375, 'GGUUAG', 138
0), (1385, 'GAC', 1387), (1388, 'CUAAG', 1392), (1394, 'CGAGG', 1398), (140
7, 'CGUAG', 1411), (1413, 'CGAUG', 1417), (1418, 'GACAA', 1422), (1446, 'UC
C', 1448), (1452, 'CCAUUUGAG', 1460), (1461, 'CAAUG', 1465), (1468, 'GGGAC
G', 1473), (1476, 'GGAG', 1479), (1481, 'AUAG', 1484), (1486, 'GUA', 1488),
(1488, 'AAG', 1490), (1492, 'GCGGUAUUG', 1500), (1500, 'GGAUAUCC', 1507), (1
509, 'CGU', 1511), (1512, 'CCAAGC', 1517), (1517, 'CAGUUA', 1522), (1523, 'G
GC', 1525), (1525, 'CUGG', 1528), (1533, 'UAG', 1535), (1548, 'CCC', 1550),
(1550, 'CAUAAG', 1555), (1555, 'GGC', 1557), (1558, 'UGAGCUG', 1564), (1564,
'GUGAUG', 1569), (1571, 'CGAG', 1574), (1575, 'CGAA', 1578), (1585, 'UAG', 1
587), (1588, 'CGAAG', 1592), (1593, 'UUC', 1595), (1597, 'UGAUU', 1601), (16
03, 'CACAC', 1607), (1614, 'GAAAAGC', 1620), (1623, 'CUAG', 1626), (1627, 'C
GAGG', 1631), (1644, 'GUA', 1646), (1648, 'CGCAAAC', 1654), (1663, 'GUAG', 1
666), (1668, 'CGA', 1670), (1682, 'UAAG', 1685), (1688, 'GAUC', 1691), (169
4, 'GAGAAC', 1699), (1706, 'UAA', 1708), (1709, 'GGAA', 1712), (1712, 'ACUCG
G', 1717), (1717, 'GCAAAAUGA', 1725), (1729, 'CGUAAC', 1734), (1734, 'CUU',
1736), (1736, 'UCGG', 1739), (1739, 'GGAGAAG', 1745), (1754, 'UGUUAG', 175
9), (1770, 'CGAGA', 1774), (1783, 'GUGAAUAGG', 1791), (1799, 'GAC', 1801),
(1801, 'CUG', 1803), (1815, 'CACAG', 1819), (1825, 'UGCGAAG', 1831), (1840,
'CGAAGUAUA', 1848), (1854, 'UGA', 1856), (1860, 'CUG', 1862), (1864, 'CCGG',
1867), (1873, 'GAAG', 1876), (1878, 'UUAAG', 1882), (1883, 'AGGAG', 1887),
(1891, 'UUAG', 1894), (1901, 'CGAAG', 1905), (1909, 'CGAAU', 1913), (1914,
'UGAAG', 1918), (1925, 'UAAA', 1928), (1951, 'CCUAAGGUAG', 1960), (1961, 'CG
A', 1963), (1970, 'UUG', 1972), (1988, 'CCCG', 1991), (1991, 'GCAC', 1994),
(1994, 'CGAAAG', 1999), (2002, 'GCAAC', 2006), (2007, 'GAU', 2009), (2015,
'CAC', 2017), (2017, 'CUGU', 2020), (2022, 'UCA', 2024), (2031, 'GAC', 203
3), (2038, 'UGAAAUU', 2044), (2045, 'AUA', 2047), (2047, 'AGUA', 2050), (205
4, 'GUGAAGA', 2060), (2061, 'UGC', 2063), (2067, 'UUAC', 2070), (2077, 'CA
G', 2079), (2081, 'ACG', 2083), (2084, 'GAAAGACC', 2091), (2094, 'GUG', 209
6), (2101, 'UUUA', 2104), (2108, 'CAG', 2110), (2118, 'UUG', 2120), (2122,
'AUG', 2124), (2125, 'UUGGUA', 2130), (2134, 'CUUG', 2137), (2137, 'GUACAG
G', 2143), (2143, 'GAUAG', 2147), (2148, 'GUA', 2150), (2150, 'AGGAG', 215
4), (2154, 'GCC', 2156), (2156, 'CUUGGAAACCGGAGC', 2170), (2175, 'GCUUUGGUGG
AGG', 2187), (2188, 'CAUC', 2191), (2191, 'CGGUGGGAUACU', 2202), (2202, 'UA
C', 2204), (2206, 'CUG', 2208), (2208, 'GGC', 2210), (2213, 'UAUUGA', 2218),
(2219, 'CCU', 2221), (2223, 'CUAACC', 2228), (2229, 'CGC', 2231), (2249, 'GA
```

```
GACAG', 2255), (2263, 'UGGG', 2266), (2270, 'UUUG', 2273), (2275, 'CUGGGGC
G', 2282), (2291, 'CUAAAAGGUAAC', 2302), (2312, 'CAAAG', 2316), (2322, 'CU
C', 2324), (2324, 'CAG', 2326), (2344, 'CGCAG', 2348), (2348, 'GAG', 2350),
(2353, 'UAA', 2355), (2359, 'CACAAG', 2364), (2370, 'UUGAC', 2374), (2377,
'CGAG', 2380), (2382, 'CCUAC', 2386), (2386, 'CAAG', 2389), (2391, 'CGAG', 2
394), (2397, 'GGG', 2399), (2408, 'CGG', 2410), (2414, 'UAG', 2416), (2416,
'GUGAUCC', 2422), (2428, 'UUC', 2430), (2431, 'CGCA', 2434), (2437, 'GAAG',
2440), (2448, 'GCUCAAC', 2454), (2455, 'GGAUAAAA', 2462), (2465, 'UACC', 246
8), (2473, 'GGAUAACA', 2480), (2484, 'UUA', 2486), (2486, 'AUC', 2488), (249
3, 'CCAAG', 2497), (2507, 'CGACG', 2511), (2516, 'GGUUU', 2520), (2523, 'CA
C', 2525), (2525, 'CCU', 2527), (2527, 'UCGAUGUC', 2534), (2537, 'CUC', 253
9), (2544, 'CAUC', 2547), (2567, 'CAAG', 2570), (2572, 'GUU', 2574), (2588,
'AUUAAAG', 2594), (2598, 'UACG', 2601), (2601, 'GCGAG', 2605), (2607, 'UGG',
2609), (2609, 'GGUUCAG', 2615), (2628, 'CAG', 2630), (2633, 'CGGU', 2636),
(2636, 'UCC', 2638), (2639, 'CUAU', 2642), (2653, 'CGCAG', 2657), (2663, 'UU
GAG', 2667), (2670, 'GAGC', 2673), (2679, 'UUAGUAC', 2685), (2690, 'GGACCG',
2695), (2701, 'GAC', 2703), (2707, 'CCGCU', 2711), (2714, 'UGUAC', 2718), (2
728, 'UGC', 2730), (2730, 'CCAA', 2733), (2738, 'AUCG', 2741), (2745, 'GGUA
G', 2749), (2752, 'AUG', 2754), (2758, 'GGACG', 2762), (2764, 'GAUAAG', 276
9), (2772, 'CUGAAAG', 2778), (2778, 'GCAUCUAAG', 2786), (2789, 'UGAAGC', 279
4), (2796, 'CCC', 2798), (2801, 'CAAG', 2804), (2804, 'GAUGAG', 2809), (281
6, 'CAU', 2818), (2829, 'AGUAAG', 2834), (2854, 'GUUGAU', 2859), (2866, 'GA
G', 2868), (2870, 'UGGAAG', 2875), (2888, 'UGGAG', 2892), (2894, 'UGAC', 289
7), (2901, 'UAC', 2903), (2905, 'AAUCG', 2909), (2912, 'CGAGG', 2916), (292
2, 'ACC', 2924)]
3Q3Z-V: wrong
test:
[(1, 'gC', 2), (6, 'GAAA', 9), (10, 'CAAUGAUG', 17), (25, 'UUAAAUU', 31), (3
8, 'UUG', 40), (48, 'UUGAG', 52), (53, 'UUAGUA', 58), (63, 'AAC', 65), (66,
'CGAC', 69), (74, 'CU', 75)]
correct:
[(1, 'GCGCGG', 6), (6, 'GAAA', 9), (10, 'CAAUGAUG', 17), (25, 'UUAAAUU', 3
1), (38, 'UUG', 40), (48, 'UUGAG', 52), (53, 'UUAGUA', 58), (63, 'AAC', 65),
(66, 'CGAC', 69), (74, 'CU', 75)]
1ET4-A: correct
6ERI-BA: correct
5O60-A: wrong
test:
[(1, 'AA', 2), (4, 'UGUUUAAG', 11), (20, 'GUGGAUG', 26), (28, 'CUUG', 31),
(43, 'CGAUGAAG', 50), (55, 'UAG', 57), (58, 'GAG', 60), (62, 'CUG', 64), (6
6, 'GAUAAGC', 72), (77, 'GGGAG', 81), (86, 'CAAC', 89), (90, 'CGA', 92), (9
4, 'CGUUGAUC', 101), (106, 'GAUGU', 110), (112, 'CGAAUG', 117), (146, 'CAC',
148), (174, 'GGG', 176), (177, 'GGAAC', 181), (186, 'GGAAGUG', 192), (192,
'GAAAC', 196), (196, 'CAUC', 199), (199, 'CUCAGUAC', 206), (210, 'UAG', 21
2), (212, 'GGAAGAGAAAA', 222), (230, 'UGAUU', 234), (238, 'UGAGUAG', 244),
(245, 'UGG', 247), (247, 'GCGAG', 251), (252, 'CGAAAG', 257), (261, 'AGGAU
G', 266), (269, 'UAAAC', 273), (282, 'UGUGAUACCG', 291), (296, 'GGGGUUG', 30
2), (311, 'GGGUUG', 316), (316, 'GUG', 318), (321, 'ACCU', 324), (324, 'UAUC
U', 328), (328, 'UUU', 330), (349, 'AGG', 351), (355, 'GUGAGAAAA', 363), (36
8, 'GUG', 370), (373, 'UAG', 375), (377, 'GGAAA', 381), (381, 'AUG', 383),
(404, 'GAC', 406), (408, 'GUGAGAG', 414), (418, 'GGUAC', 422), (424, 'UGAA
A', 428), (442, 'CUUG', 445), (445, 'GAUGG', 449), (449, 'GUGUU', 453), (45
7, 'GAGUAG', 462), (468, 'GGCCC', 472), (472, 'CGUGGAAUC', 480), (486, 'UGAA
UC', 491), (491, 'CUG', 493), (498, 'GAC', 500), (508, 'UAAG', 511), (513,
'CUGAAUAC', 520), (521, 'UUC', 523), (529, 'GAC', 531), (532, 'CGAUAGCGGAUUA
GUAC', 548), (555, 'GGAAUG', 560), (561, 'GUGAAAAG', 568), (568, 'GUAC', 57
```

```
1), (582, 'GAGUGAAAGAGUAC', 595), (596, 'CUGAAAC', 602), (611, 'UACAAUC', 61
7), (617, 'CCG', 619), (622, 'AGAG', 625), (628, 'CUC', 630), (639, 'GUGG',
642), (645, 'UGAU', 648), (651, 'CGU', 653), (653, 'UGC', 655), (655, 'CCU',
657), (666, 'AUG', 668), (668, 'GAG', 670), (675, 'CGAG', 678), (678, 'GUC',
680), (682, 'GGG', 684), (693, 'CGAG', 696), (698, 'UUAAC', 702), (711, 'GGU
AG', 715), (717, 'CGCAG', 721), (728, 'CGAG', 731), (743, 'CGUAUC', 748), (7
65, 'GUGUAG', 770), (782, 'GGAC', 785), (786, 'CCG', 788), (788, 'GAAGC', 79
2), (797, 'UGAUCUA', 803), (812, 'CAG', 814), (817, 'UGAAG', 821), (838, 'UG
GAG', 842), (846, 'CGAAC', 850), (852, 'CAC', 854), (858, 'GGUUGAAGA', 866),
(874, 'GAUG', 877), (877, 'GAG', 879), (887, 'UAG', 889), (889, 'GGG', 891),
(901, 'CAAUC', 905), (905, 'CAAA', 908), (913, 'GUGAUAGC', 920), (920, 'CU
G', 922), (924, 'UUC', 926), (931, 'CGAAAU', 936), (958, 'UGU', 960), (961,
'UUC', 963), (970, 'GAGGUAG', 976), (978, 'GCUACU', 983), (985, 'GAU', 987),
(990, 'CCGAUG', 995), (995, 'GGGC', 998), (1007, 'GUUAC', 1011), (1011, 'CUG
ACG', 1016), (1019, 'AGC', 1021), (1023, 'AAAC', 1026), (1027, 'UCC', 1029),
(1029, 'CGAAUGC', 1035), (1042, 'GUCCAA', 1047), (1047, 'AGA', 1049), (1056,
'CAG', 1058), (1059, 'UGAG', 1062), (1083, 'UGC', 1085), (1088, 'CGAGAG', 10
93), (1104, 'CAGAU', 1108), (1111, 'CCG', 1113), (1115, 'CUAAG', 1119), (112
4, 'CUAAG', 1128), (1128, 'GCG', 1130), (1135, 'CUAAGUG', 1141), (1141, 'GGA
AAAG', 1147), (1148, 'GAUG', 1151), (1155, 'AGU', 1157), (1160, 'CGAAGAC', 1
166), (1166, 'CAA', 1168), (1172, 'GGA', 1174), (1177, 'UUG', 1179), (1193,
'CACC', 1196), (1198, 'UUGAAAG', 1204), (1205, 'AGUG', 1208), (1217, 'CUC',
1219), (1219, 'CAC', 1221), (1225, 'UCAAG', 1229), (1232, 'AUU', 1234), (123
8, 'CGC', 1240), (1241, 'CGAUAAUGUAGCG', 1253), (1255, 'GGC', 1257), (1257,
'CUCAAG', 1262), (1271, 'CGAAG', 1275), (1299, 'GGGUAG', 1304), (1310, 'CG
U', 1312), (1321, 'CGGUGAAG', 1328), (1342, 'GAG', 1344), (1348, 'UGGAG', 13
52), (1361, 'AGUG', 1364), (1364, 'GAGAAUG', 1370), (1376, 'AUGAGUAG', 138
3), (1384, 'CGAUUAG', 1390), (1412, 'CGAAAG', 1417), (1420, 'CAAG', 1423),
(1424, 'GGUU', 1427), (1433, 'GCCAG', 1437), (1445, 'CGC', 1447), (1452, 'GG
UGAG', 1457), (1462, 'GAC', 1464), (1465, 'CUA', 1467), (1467, 'AAG', 1469),
(1471, 'CGAGG', 1475), (1484, 'CGUAG', 1488), (1490, 'CGAU', 1493), (1495,
'GACAA', 1499), (1528, 'UGC', 1530), (1530, 'CGUCCAU', 1536), (1539, 'UGAA',
1542), (1549, 'GUACUAAC', 1556), (1559, 'UCC', 1561), (1562, 'AAAAC', 1566),
(1572, 'UGAC', 1575), (1576, 'CGC', 1578), (1591, 'GUG', 1593), (1594, 'GC
G', 1596), (1597, 'UUG', 1599), (1603, 'GGGCU', 1607), (1608, 'GCA', 1610),
(1613, 'GGACCUU', 1619), (1626, 'UAGUAG', 1631), (1634, 'AAG', 1636), (1637,
'CGAUG', 1641), (1645, 'UGACG', 1649), (1652, 'GGAA', 1655), (1657, 'GUAG',
1660), (1663, 'GUAC', 1666), (1670, 'UCAGUG', 1675), (1676, 'GUAAUA', 1681),
(1685, 'GGG', 1687), (1688, 'UAAG', 1691), (1693, 'CUGUAG', 1698), (1708, 'U
AG', 1710), (1725, 'CAUAUAU', 1731), (1734, 'UGAG', 1737), (1737, 'GAG', 173
9), (1741, 'UGAUG', 1745), (1746, 'CAUAG', 1750), (1761, 'CGAAUUC', 1767),
(1770, 'UGAUC', 1774), (1776, 'UAUGC', 1780), (1787, 'GAAAAG', 1792), (1796,
'CUAG', 1799), (1800, 'CGAG', 1803), (1803, 'GGA', 1805), (1818, 'GUA', 182
0), (1822, 'CCCAAAC', 1828), (1837, 'GUG', 1839), (1842, 'CAG', 1844), (185
6, 'UAAG', 1859), (1862, 'GUAC', 1865), (1868, 'GUGAAC', 1873), (1880, 'UA
A', 1882), (1883, 'GGAA', 1886), (1886, 'ACUCGG', 1891), (1891, 'GCAAAAUGC',
1899), (1903, 'CGUAAC', 1908), (1908, 'CUU', 1910), (1910, 'UCGG', 1913), (1
913, 'GGAGAAG', 1919), (1929, 'CAUGG', 1933), (1936, 'UGUAAG', 1941), (1951,
'CCCAAG', 1956), (1959, 'UGAG', 1962), (1972, 'CAAACCAGU', 1980), (1988, 'GA
C', 1990), (1990, 'CUG', 1992), (2004, 'CACAG', 2008), (2014, 'UGCGAAG', 202
0), (2029, 'CGAUGUAUA', 2037), (2043, 'UGA', 2045), (2049, 'CUG', 2051), (20
53, 'CCGG', 2056), (2062, 'GAAG', 2065), (2067, 'UUAAG', 2071), (2072, 'AGGA
CC', 2077), (2080, 'UUAA', 2083), (2097, 'UGAAG', 2101), (2104, 'GAGAAU', 21
09), (2110, 'UUAAG', 2114), (2121, 'UAAA', 2124), (2147, 'CCUAAGGUAG', 215
6), (2157, 'CGAAAUUCCUUG', 2168), (2184, 'CCUG', 2187), (2187, 'GCAC', 219
0), (2190, 'CGAAUG', 2195), (2198, 'GUAAC', 2202), (2203, 'GAC', 2205), (221
```

```
1, 'AAC', 2213), (2213, 'CUGU', 2216), (2218, 'UCA', 2220), (2227, 'GAC', 22
29), (2234, 'CGAAAUU', 2240), (2243, 'ACUA', 2246), (2250, 'GUAAAGA', 2256),
(2263, 'UUAC', 2266), (2273, 'CAG', 2275), (2277, 'ACG', 2279), (2280, 'AAAA
GACC', 2287), (2296, 'UUCA', 2299), (2303, 'CAA', 2305), (2307, 'UUG', 230
9), (2313, 'UUG', 2315), (2316, 'GUG', 2318), (2329, 'UUUG', 2332), (2332,
'GUGUAGGAUAGG', 2343), (2343, 'GUG', 2345), (2346, 'GGAGA', 2350), (2351, 'C
UGUGAAG', 2358), (2359, 'CUC', 2361), (2374, 'GGG', 2376), (2377, 'UGGAG', 2
381), (2382, 'UCGUUGUUGAAAUAC', 2396), (2397, 'CAC', 2399), (2399, 'CUC', 24
01), (2401, 'CUGA', 2404), (2415, 'CCU', 2417), (2418, 'CUAAC', 2422), (242
4, 'UCG', 2426), (2446, 'GGACAG', 2451), (2455, 'CUG', 2457), (2459, 'UGGG',
2462), (2466, 'UUUA', 2469), (2471, 'CUGGGGCG', 2478), (2487, 'CUAAAAUGUAA
C', 2498), (2508, 'CAAAG', 2512), (2518, 'CUC', 2520), (2520, 'CAA', 2522),
(2540, 'UGUUG', 2544), (2544, 'GAG', 2546), (2549, 'UAA', 2551), (2555, 'CAC
AAG', 2560), (2566, 'UUGAC', 2570), (2573, 'CGAG', 2576), (2578, 'CGGAC', 25
82), (2582, 'CAUG', 2585), (2587, 'CGAG', 2590), (2593, 'GGG', 2595), (2604,
'CGG', 2606), (2610, 'UAG', 2612), (2612, 'GUGAUCC', 2618), (2624, 'CUC', 26
26), (2627, 'UGAG', 2630), (2633, 'GAAG', 2636), (2644, 'GCUCAAC', 2650), (2
651, 'GGAUAAAA', 2658), (2661, 'UACC', 2664), (2669, 'GGAUAACA', 2676), (268
0, 'UGA', 2682), (2682, 'AUC', 2684), (2684, 'CUU', 2686), (2689, 'CCAAG', 2
693), (2703, 'CGACG', 2707), (2710, 'AUG', 2712), (2712, 'GGUUU', 2716), (27
19, 'CAC', 2721), (2721, 'CCU', 2723), (2723, 'UCGAUGU', 2729), (2733, 'CU
C', 2735), (2740, 'CAUC', 2743), (2763, 'CAAG', 2766), (2768, 'GUU', 2770),
(2784, 'AUUAAAG', 2790), (2794, 'CACG', 2797), (2797, 'GCGAG', 2801), (2803,
'UGG', 2805), (2806, 'GUUUAG', 2811), (2824, 'CAG', 2826), (2829, 'CGGU', 28
32), (2832, 'UCU', 2834), (2835, 'CUAU', 2838), (2849, 'CGUCA', 2853), (285
9, 'UUGAG', 2863), (2866, 'AACC', 2869), (2875, 'CUAGUAC', 2881), (2886, 'GG
ACCG', 2891), (2897, 'GAC', 2899), (2903, 'CCUCU', 2907), (2910, 'UAUAC', 29
14), (2924, 'CAC', 2926), (2927, 'CAG', 2929), (2934, 'ACGG', 2937), (2941,
'GAUAG', 2945), (2948, 'ACG', 2950), (2954, 'GGACA', 2958), (2960, 'GAUAAC',
2965), (2968, 'CUGAAAG', 2974), (2974, 'GCAUCUAAG', 2982), (2985, 'GGAAAC',
2990), (2992, 'UCU', 2994), (2997, 'CAAG', 3000), (3000, 'GACCAG', 3005), (3
011, 'UCAC', 3014), (3026, 'GAUAAG', 3031), (3034, 'CCC', 3036), (3050, 'GAU
UGAU', 3056), (3058, 'GAC', 3060), (3063, 'GAC', 3065), (3067, 'UGGAAG', 307
2), (3085, 'UGCAG', 3089), (3091, 'GAAC', 3094), (3097, 'GCAC', 3100), (310
2, 'AACCG', 3106), (3109, 'CGAAAA', 3114), (3117, 'UAC', 3119)]
correct:
[(1, 'AAGU', 4), (4, 'UGUUUAAG', 11), (20, 'GUGGAUG', 26), (28, 'CUUG', 31),
(43, 'CGAUGAAG', 50), (55, 'UAG', 57), (58, 'GAG', 60), (62, 'CUG', 64), (6
6, 'GAUAAGC', 72), (77, 'GGGAG', 81), (86, 'CAAC', 89), (90, 'CGA', 92), (9
4, 'CGUUGAUC', 101), (106, 'GAUGU', 110), (112, 'CGAAUG', 117), (146, 'CAC',
148), (174, 'GGG', 176), (177, 'GGAAC', 181), (186, 'GGAAGUG', 192), (192,
'GAAAC', 196), (196, 'CAUC', 199), (199, 'CUCAGUAC', 206), (210, 'UAG', 21
2), (212, 'GGAAGAGAAAA', 222), (230, 'UGAUU', 234), (238, 'UGAGUAG', 244),
(245, 'UGG', 247), (247, 'GCGAG', 251), (252, 'CGAAAG', 257), (261, 'AGGAU
G', 266), (269, 'UAAAC', 273), (282, 'UGUGAUACCG', 291), (296, 'GGGGUUG', 30
2), (311, 'GGGUUG', 316), (316, 'GUG', 318), (321, 'ACCU', 324), (324, 'UAUC
U', 328), (328, 'UUU', 330), (349, 'AGG', 351), (355, 'GUGAGAAAA', 363), (36
8, 'GUG', 370), (373, 'UAG', 375), (377, 'GGAAA', 381), (381, 'AUG', 383),
(404, 'GAC', 406), (408, 'GUGAGAG', 414), (418, 'GGUAC', 422), (424, 'UGAA
A', 428), (442, 'CUUG', 445), (445, 'GAUGG', 449), (449, 'GUGUU', 453), (45
7, 'GAGUAG', 462), (468, 'GGCCC', 472), (472, 'CGUGGAAUC', 480), (486, 'UGAA
UC', 491), (491, 'CUG', 493), (498, 'GAC', 500), (508, 'UAAG', 511), (513,
'CUGAAUAC', 520), (521, 'UUC', 523), (529, 'GAC', 531), (532, 'CGAUAGCGGAUUA
GUAC', 548), (555, 'GGAAUG', 560), (561, 'GUGAAAAG', 568), (568, 'GUAC', 57
1), (582, 'GAGUGAAAGAGUAC', 595), (596, 'CUGAAAC', 602), (611, 'UACAAUC', 61
7), (617, 'CCG', 619), (622, 'AGAG', 625), (628, 'CUC', 630), (639, 'GUGG',
```

```
642), (645, 'UGAU', 648), (651, 'CGU', 653), (653, 'UGC', 655), (655, 'CCU',
657), (666, 'AUG', 668), (668, 'GAG', 670), (675, 'CGAG', 678), (678, 'GUC',
680), (682, 'GGG', 684), (693, 'CGAG', 696), (698, 'UUAAC', 702), (711, 'GGU
AG', 715), (717, 'CGCAG', 721), (728, 'CGAG', 731), (743, 'CGUAUC', 748), (7
65, 'GUGUAG', 770), (782, 'GGAC', 785), (786, 'CCG', 788), (788, 'GAAGC', 79
2), (797, 'UGAUCUA', 803), (812, 'CAG', 814), (817, 'UGAAG', 821), (838, 'UG
GAG', 842), (846, 'CGAAC', 850), (852, 'CAC', 854), (858, 'GGUUGAAGA', 866),
(874, 'GAUG', 877), (877, 'GAG', 879), (887, 'UAG', 889), (889, 'GGG', 891),
(901, 'CAAUC', 905), (905, 'CAAA', 908), (913, 'GUGAUAGC', 920), (920, 'CU
G', 922), (924, 'UUC', 926), (931, 'CGAAAU', 936), (958, 'UGU', 960), (961,
'UUC', 963), (970, 'GAGGUAG', 976), (978, 'GCUACU', 983), (985, 'GAU', 987),
(990, 'CCGAUG', 995), (995, 'GGGC', 998), (1007, 'GUUAC', 1011), (1011, 'CUG
ACG', 1016), (1019, 'AGC', 1021), (1023, 'AAAC', 1026), (1027, 'UCC', 1029),
(1029, 'CGAAUGC', 1035), (1042, 'GUCCAA', 1047), (1047, 'AGA', 1049), (1056,
'CAG', 1058), (1059, 'UGAG', 1062), (1083, 'UGC', 1085), (1088, 'CGAGAG', 10
93), (1104, 'CAGAU', 1108), (1111, 'CCG', 1113), (1115, 'CUAAG', 1119), (112
4, 'CUAAG', 1128), (1128, 'GCG', 1130), (1135, 'CUAAGUG', 1141), (1141, 'GGA
AAAG', 1147), (1148, 'GAUG', 1151), (1155, 'AGU', 1157), (1160, 'CGAAGAC', 1
166), (1166, 'CAA', 1168), (1172, 'GGA', 1174), (1177, 'UUG', 1179), (1193,
'CACC', 1196), (1198, 'UUGAAAG', 1204), (1205, 'AGUG', 1208), (1217, 'CUC',
1219), (1219, 'CAC', 1221), (1225, 'UCAAG', 1229), (1232, 'AUU', 1234), (123
8, 'CGC', 1240), (1241, 'CGAUAAUGUAGCG', 1253), (1255, 'GGC', 1257), (1257,
'CUCAAG', 1262), (1271, 'CGAAG', 1275), (1299, 'GGGUAG', 1304), (1310, 'CG
U', 1312), (1321, 'CGGUGAAG', 1328), (1342, 'GAG', 1344), (1348, 'UGGAG', 13
52), (1361, 'AGUG', 1364), (1364, 'GAGAAUG', 1370), (1376, 'AUGAGUAG', 138
3), (1384, 'CGAUUAG', 1390), (1412, 'CGAAAG', 1417), (1420, 'CAAG', 1423),
(1424, 'GGUU', 1427), (1433, 'GCCAG', 1437), (1445, 'CGC', 1447), (1452, 'GG
UGAG', 1457), (1462, 'GAC', 1464), (1465, 'CUA', 1467), (1467, 'AAG', 1469),
(1471, 'CGAGG', 1475), (1484, 'CGUAG', 1488), (1490, 'CGAU', 1493), (1495,
'GACAA', 1499), (1528, 'UGC', 1530), (1530, 'CGUCCAU', 1536), (1539, 'UGAA',
1542), (1549, 'GUACUAAC', 1556), (1559, 'UCC', 1561), (1562, 'AAAAC', 1566),
(1572, 'UGAC', 1575), (1576, 'CGC', 1578), (1591, 'GUG', 1593), (1594, 'GC
G', 1596), (1597, 'UUG', 1599), (1603, 'GGGCU', 1607), (1608, 'GCA', 1610),
(1613, 'GGACCUU', 1619), (1626, 'UAGUAG', 1631), (1634, 'AAG', 1636), (1637,
'CGAUG', 1641), (1645, 'UGACG', 1649), (1652, 'GGAA', 1655), (1657, 'GUAG',
1660), (1663, 'GUAC', 1666), (1670, 'UCAGUG', 1675), (1676, 'GUAAUA', 1681),
(1685, 'GGG', 1687), (1688, 'UAAG', 1691), (1693, 'CUGUAG', 1698), (1708, 'U
AG', 1710), (1725, 'CAUAUAU', 1731), (1734, 'UGAG', 1737), (1737, 'GAG', 173
9), (1741, 'UGAUG', 1745), (1746, 'CAUAG', 1750), (1761, 'CGAAUUC', 1767),
(1770, 'UGAUC', 1774), (1776, 'UAUGC', 1780), (1787, 'GAAAAG', 1792), (1796,
'CUAG', 1799), (1800, 'CGAG', 1803), (1803, 'GGA', 1805), (1818, 'GUA', 182
0), (1822, 'CCCAAAC', 1828), (1837, 'GUG', 1839), (1842, 'CAG', 1844), (185
6, 'UAAG', 1859), (1862, 'GUAC', 1865), (1868, 'GUGAAC', 1873), (1880, 'UA
A', 1882), (1883, 'GGAA', 1886), (1886, 'ACUCGG', 1891), (1891, 'GCAAAAUGC',
1899), (1903, 'CGUAAC', 1908), (1908, 'CUU', 1910), (1910, 'UCGG', 1913), (1
913, 'GGAGAAG', 1919), (1929, 'CAUGG', 1933), (1936, 'UGUAAG', 1941), (1951,
'CCCAAG', 1956), (1959, 'UGAG', 1962), (1972, 'CAAACCAGU', 1980), (1988, 'GA
C', 1990), (1990, 'CUG', 1992), (2004, 'CACAG', 2008), (2014, 'UGCGAAG', 202
0), (2029, 'CGAUGUAUA', 2037), (2043, 'UGA', 2045), (2049, 'CUG', 2051), (20
53, 'CCGG', 2056), (2062, 'GAAG', 2065), (2067, 'UUAAG', 2071), (2072, 'AGGA
CC', 2077), (2080, 'UUAA', 2083), (2097, 'UGAAG', 2101), (2104, 'GAGAAU', 21
09), (2110, 'UUAAG', 2114), (2121, 'UAAA', 2124), (2147, 'CCUAAGGUAG', 215
6), (2157, 'CGAAAUUCCUUG', 2168), (2184, 'CCUG', 2187), (2187, 'GCAC', 219
0), (2190, 'CGAAUG', 2195), (2198, 'GUAAC', 2202), (2203, 'GAC', 2205), (221
1, 'AAC', 2213), (2213, 'CUGU', 2216), (2218, 'UCA', 2220), (2227, 'GAC', 22
29), (2234, 'CGAAAUU', 2240), (2243, 'ACUA', 2246), (2250, 'GUAAAGA', 2256),
```

```
(2263, 'UUAC', 2266), (2273, 'CAG', 2275), (2277, 'ACG', 2279), (2280, 'AAAA
GACC', 2287), (2296, 'UUCA', 2299), (2303, 'CAA', 2305), (2307, 'UUG', 230
9), (2313, 'UUG', 2315), (2316, 'GUG', 2318), (2329, 'UUUG', 2332), (2332,
'GUGUAGGAUAGG', 2343), (2343, 'GUG', 2345), (2346, 'GGAGA', 2350), (2351, 'C
UGUGAAG', 2358), (2359, 'CUC', 2361), (2374, 'GGG', 2376), (2377, 'UGGAG', 2
381), (2382, 'UCGUUGUUGAAAUAC', 2396), (2397, 'CAC', 2399), (2399, 'CUC', 24
01), (2401, 'CUGA', 2404), (2415, 'CCU', 2417), (2418, 'CUAAC', 2422), (242
4, 'UCG', 2426), (2446, 'GGACAG', 2451), (2455, 'CUG', 2457), (2459, 'UGGG',
2462), (2466, 'UUUA', 2469), (2471, 'CUGGGGCG', 2478), (2487, 'CUAAAAUGUAA
C', 2498), (2508, 'CAAAG', 2512), (2518, 'CUC', 2520), (2520, 'CAA', 2522),
(2540, 'UGUUG', 2544), (2544, 'GAG', 2546), (2549, 'UAA', 2551), (2555, 'CAC
AAG', 2560), (2566, 'UUGAC', 2570), (2573, 'CGAG', 2576), (2578, 'CGGAC', 25
82), (2582, 'CAUG', 2585), (2587, 'CGAG', 2590), (2593, 'GGG', 2595), (2604,
'CGG', 2606), (2610, 'UAG', 2612), (2612, 'GUGAUCC', 2618), (2624, 'CUC', 26
26), (2627, 'UGAG', 2630), (2633, 'GAAG', 2636), (2644, 'GCUCAAC', 2650), (2
651, 'GGAUAAAA', 2658), (2661, 'UACC', 2664), (2669, 'GGAUAACA', 2676), (268
0, 'UGA', 2682), (2682, 'AUC', 2684), (2684, 'CUU', 2686), (2689, 'CCAAG', 2
693), (2703, 'CGACG', 2707), (2710, 'AUG', 2712), (2712, 'GGUUU', 2716), (27
19, 'CAC', 2721), (2721, 'CCU', 2723), (2723, 'UCGAUGU', 2729), (2733, 'CU
C', 2735), (2740, 'CAUC', 2743), (2763, 'CAAG', 2766), (2768, 'GUU', 2770),
(2784, 'AUUAAAG', 2790), (2794, 'CACG', 2797), (2797, 'GCGAG', 2801), (2803,
'UGG', 2805), (2806, 'GUUUAG', 2811), (2824, 'CAG', 2826), (2829, 'CGGU', 28
32), (2832, 'UCU', 2834), (2835, 'CUAU', 2838), (2849, 'CGUCA', 2853), (285
9, 'UUGAG', 2863), (2866, 'AACC', 2869), (2875, 'CUAGUAC', 2881), (2886, 'GG
ACCG', 2891), (2897, 'GAC', 2899), (2903, 'CCUCU', 2907), (2910, 'UAUAC', 29
14), (2924, 'CAC', 2926), (2927, 'CAG', 2929), (2934, 'ACGG', 2937), (2941,
'GAUAG', 2945), (2948, 'ACG', 2950), (2954, 'GGACA', 2958), (2960, 'GAUAAC',
2965), (2968, 'CUGAAAG', 2974), (2974, 'GCAUCUAAG', 2982), (2985, 'GGAAAC',
2990), (2992, 'UCU', 2994), (2997, 'CAAG', 3000), (3000, 'GACCAG', 3005), (3
011, 'UCAC', 3014), (3026, 'GAUAAG', 3031), (3034, 'CCC', 3036), (3050, 'GAU
UGAU', 3056), (3058, 'GAC', 3060), (3063, 'GAC', 3065), (3067, 'UGGAAG', 307
2), (3085, 'UGCAG', 3089), (3091, 'GAAC', 3094), (3097, 'GCAC', 3100), (310
2, 'AACCG', 3106), (3109, 'CGAAAA', 3114), (3117, 'UAC', 3119)]
3IVN-A: wrong
test:
[(1, 'GG', 2), (9, 'UAAC', 12), (17, 'AUGAUAUG', 24), (25, 'GUU', 27), (33,
'GUG', 35), (35, 'GUCUA', 39), (45, 'AAC', 47), (48, 'CGUAAAAU', 55), (59,
'GAC', 61)]
correct:
[(1, 'GGCCAGUAU', 9), (9, 'UAAC', 12), (17, 'AUGAUAUG', 24), (25, 'GUU', 2
7), (33, 'GUG', 35), (35, 'GUCUA', 39), (45, 'AAC', 47), (48, 'CGUAAAAU', 5
5), (59, 'GAC', 61), (61, 'CUACUGGUC', 69)]
6N5P-A: wrong
test:
[(1, 'gGUU', 4), (7, 'CGAAUC', 12), (24, 'GUAC', 27), (31, 'GGAACC', 36), (3
6, 'CGCUUUU', 42), (42, 'UUG', 44), (48, 'UUAAUC', 53), (69, 'GUAG', 72), (7
6, 'UAC', 78), (88, 'CGCA', 91), (94, 'CGA', 96), (97, 'CAG', 99), (99, 'GCU
AAC', 104), (108, 'GGAG', 111), (114, 'AAUAAAG', 120), (125, 'GAG', 127)]
correct:
[(1, 'GGUU', 4), (7, 'CGAAUC', 12), (24, 'GUAC', 27), (31, 'GGAACC', 36), (3
6, 'CGCUUUU', 42), (42, 'UUG', 44), (48, 'UUAAUC', 53), (69, 'GUAG', 72), (7
6, 'UAC', 78), (88, 'CGCA', 91), (94, 'CGA', 96), (97, 'CAG', 99), (99, 'GCU
AAC', 104), (108, 'GGAG', 111), (114, 'AAUAAAG', 120), (125, 'GAG', 127)]
6HA1-a: correct
6V3A-sN1: correct
6AGB-A: correct
```

```
6ZM6-AA: correct
5J7L-AA: correct
5SWD-B: wrong
test:
[(1, 'GGGA', 4), (9, 'AUAAU', 13), (18, 'AUGAUAUG', 25), (26, 'GUU', 28), (3
5, 'UUU', 37), (37, 'UCUA', 40), (49, 'CUUAAAC', 55), (61, 'AUU', 63)]
correct:
[(1, 'GGGA', 4), (9, 'AUAAU', 13), (18, 'AUGAUAUG', 25), (26, 'GUU', 28), (3
5, 'UUU', 37), (37, 'UCUA', 40), (49, 'CUUAAAC', 55), (61, 'AUU', 63), (63,
'UAUCUU', 68)]
4FRN-B: wrong
test:
[(1, 'GG', 2), (6, 'AAAAGC', 11), (11, 'CAUG', 14), (18, 'GGAAAG', 23), (23,
'GUG', 25), (41, 'ACA', 43), (45, 'UAC', 47), (52, 'UAC', 54), (56, 'GUUAUAC
U', 63), (66, 'GAAUG', 70), (81, 'CAAAGUAG', 88), (94, 'GGAGA', 98), (101,
'CA', 102)]
correct:
[(1, 'GGGCUA', 6), (6, 'AAAAGC', 11), (11, 'CAUG', 14), (18, 'GGAAAG', 23),
(23, 'GUG', 25), (41, 'ACA', 43), (45, 'UAC', 47), (52, 'UAC', 54), (56, 'GU
UAUACU', 63), (66, 'GAAUG', 70), (81, 'CAAAGUAG', 88), (94, 'GGAGA', 98), (1
01, 'CA', 102)]
5TPY-A: wrong
test:
[(1, 'GGG', 3), (3, 'GUC', 5), (21, 'GCC', 23), (28, 'UUUG', 31), (34, 'GAAA
G', 38), (44, 'CAG', 46), (50, 'GUAAC', 54)]
correct:
[(1, 'GGG', 3), (3, 'GUC', 5), (21, 'GCC', 23), (28, 'UUUG', 31), (34, 'GAAA
G', 38), (44, 'CAG', 46), (50, 'GUAAC', 54), (67, 'GUGGG', 71)]
6GAZ-AA: correct
3IGI-A: correct
```

FCFS

Wczytanie danych wynikowych

In [210…

```python
def readDotBracketPlain(path):
    with open(path, "r") as f:
        content = f.read()
    return content[:-1]
```

Algorytm FCFS

In [211…

```python
def areConflicted(stem_a: tuple, stem_b: tuple) -> bool:
    a_start = stem_a[0][0]
    a_end = stem_a[0][1]
    b_start = stem_b[0][0]
    b_end = stem_b[0][1]

    return (a_start < b_start < a_end < b_end) or (b_start < a_start < b_enc

def FCFS(bpseq):
```

```
    stems = findStems(bpseq)
    # print(stems)
    stems = [ [ (bpseq[idx][0], bpseq[idx][2]) for idx in range(stem[0][0]-1
    # print(stems)
    stems = sorted(stems,key=lambda x: x[0][0])
    stemsOrder = [ 0 for _ in range(len(stems))]

    openingBrackets = ["(", "[", "{", "<", "A", "B"]
    closingBrackets = [")", "]", "}", ">", "a", "b"]
    # print(stemsOrder)
    for i in range(1,len(stems)):
        available = [ True for _ in openingBrackets ]

        # print(available)
        for j in  range(i):
            # print(j)
            if areConflicted(stems[i], stems[j]):
                # print("conf")
                available[stemsOrder[j]] = False

        # print(available)
        current = 0
        while available[current] == False:
            current += 1

        stemsOrder[i] = current

    result = ["."] * (len(bpseq) + 1)

    nucValues = np.array([ values[1] for values in bpseq])
    seq = "".join(nucValues)

    for stem_idx, stemPairs in enumerate(stems):
        open_br = openingBrackets[stemsOrder[stem_idx]]
        close_br = closingBrackets[stemsOrder[stem_idx]]
        for left, right in stemPairs:
            result[left] = open_br
            result[right] = close_br

    dot_bracket = "".join(result[1:])
    return f"{seq}\n{dot_bracket}"
```

Test poprawności

In [212…

```
runTests(testPathPrefix="dot-brackets-correct/correct/3.0/",
         correctPathPrefix="dot-brackets-correct/correct/5.0/",
         testExt=".bpseq",
         correctExt=".dbn",
         testImportFun=readBPSEQ,
         correctImportFun=readDotBracketPlain,
         processingFun=FCFS,
         testSpecific=None)
```

```
7K16-P: correct
6UES-A: correct
6NEQ-A: correct
3Q1Q-B: correct
6TPQ-U: correct
3Q3Z-V: correct
1ET4-A: correct
6ERI-BA: correct
5O60-A: correct
3IVN-A: correct
6N5P-A: correct
6HA1-a: correct
6V3A-sN1: correct
6AGB-A: correct
6ZM6-AA: correct
5J7L-AA: correct
5SWD-B: correct
4FRN-B: correct
5TPY-A: correct
6GAZ-AA: correct
3IGI-A: correct
```